



3SHAKE

おい、テックブログを 書け

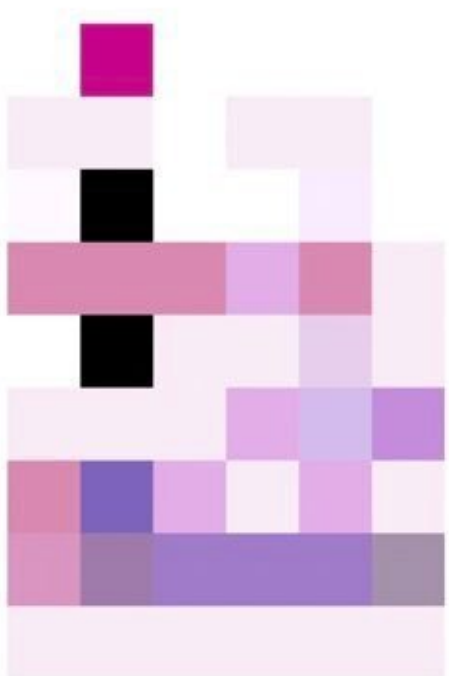
3年目までに身につけたい技術ブログの書き方

2025/12/05 Forkwell Community

おい、テックブログを書け

@nwiizo 30min

nwiizo



株式会社スリーシェイクで プロのソフトウェアエンジニアをやっているものです 格闘技、読書、グラビアが趣味でよく本を紹介しています

人生を通して"**運動、睡眠、読書**"をきちんとやりたい

<https://github.com/nwiizo>

about 3-shake

3>SHAKE

Mission

インフラをシンプルにしてイノベーションが起こりやすい世界を作る

Vision

労苦（Toil）を無くすサービスを適正な価格で提供し続ける

SRE



スリーク

日本のSREをリードする

SRE総合支援からセキュリティ対策を全方位支援

Data Engineering



レコナー

**あらゆるサービスを
連携するハブになる**

クラウド型ETL/データパイプ
ラインサービスの決定版

Security



セキュリファイ

**事業者が抱える
セキュリティリスクを無くす**

ワンストップで実現する
セキュリティ対策

HR (Engineer Hiring)



リランス

**良いエンジニアに
良い案件を**

フリーランスエンジニアに
「今よりいい条件」を



We are Hiring!!

3-shakeと一緒にSRE界隈を盛り上げてくれる**仲間を大募集中**です！

Mobility、FinTech、通信など大規模SREを存分に経験できます

(最近社内はGenAI / GPU / Kubernetesが盛り上がってます)

是非、カジュアル面談しましょう！！！！

リードSREエンジニア
SREを突き詰めたい方



SREエンジニア
SREを始めてみたい方



フルスタックエンジニア
モダンスタックでSREと開発も両方
やりたい欲張りな方



今日お話しすることfff

1. なぜ書くべきなのか

- 「書かない理由」を潰す
- 書かないコストを直視する

2. よくある失敗とAI活用

- 書けない・出せない・読まれない
- AIとの正しい付き合い方：身体性

3. 何を書くか：ネタのを見つけ方

- 日常からネタを発掘する技術
- THINK BIGGER：6ステップの発想法

4. どう書くか：記事の書き方

- 技術ブログの種類と構成
- タイトル・導入文・本文の型

5. より良い記事にするために

- 防御力の高いブログを書く
- 公開前チェックリスト

6. 実践編

- どこに・いつ書くか



この発表で解決できること

「書けない」「書かない」を終わらせる

こんな悩みを持っていませんか？

- 書こうと思っているけど、始められない
- 何を書けばいいかわからない
- 書き始めても途中で止まる
- 公開するのが怖い
- 続かない

この30分で持ち帰れるもの

- 「書くべき理由」の腹落ち
- AIとの正しい付き合い方（身体性）
- ネタを見つける6ステップの発想法
- 記事の型とチェックリスト
- 公開への心理的ハードルの下げ方

目標：この発表を聞いた人が、今週1本ブログを書く



なぜ技術ブログを書くべきなのか

「書かない理由」を全部潰す

× 「時間がない」

→ 30分の苦労を記事にすれば、100人の30分を救う。書かない方が時間の無駄

× 「もう誰かが書いている」

→ 同じ技術でも、環境・前提・視点が違う。**あなたの言葉**で救われる人がいる

× 「間違ったことを書くのが怖い」

→ 間違いは指摘されて直せばいい。**書かなければ永遠に間違いに気づけない**

× 「自分なんてまだ経験が浅い」

→ 初心者の「わからない→わかった」は、専門家には書けない。**鮮度が価値**



「書かない」という選択のコスト

失われるものを直視する

学習効果の損失

読んだだけ、やっただけでは定着しない。書くことで初めて「自分の知識」になる。書かなければ、3ヶ月後には忘れる

同じ説明を繰り返すコスト

チームで何度も同じことを説明していないか？記事にすれば1回で済む。書かないから同じ苦労を繰り返す

存在証明の機会損失

GitHubは「何を作ったか」を示す。ブログは「どう考えたか」を示す。思考が見えないエンジニアは選ばれない

コミュニティへの負債

あなたも誰かの記事に救われた。受け取るだけで返さないのはフリーライダー。書いて返せ

技術ブログ執筆がもたらす3つの価値

書くことの最大の受益者は、書いた本人だ

理解の穴が見える

曖昧に理解していたつもりが、言語化しようとした瞬間に「説明できない」と気づく

- 調べ直し、考え直し、書ける
- **理解は確実に深まる**

知識が構造化される

散らばった知識が、書くことで整理される。「なんとなく分かった」が「説明できる」に変わる

- **半年後の自分が助かる**

継続の原動力になる

他者に読まれなくても、この価値は消えない。「書いたことで自分は確実に賢くなった」

- **他者の評価に依存しない**

他者への価値だけを見ていないか

PV・はてブ・「参考になりました」だけが価値？

他者からの反応を価値の尺度にしすぎると

- PVが伸びないと落ち込む
- 反応がないと書く気が失せる
- 「誰も読まないかも」で手が止まる

→ 継続できなくなる

自分への価値を軸にすると

- 書くこと自体が報酬になる
- 反応がなくても価値がある
- 「自分のため」だからハードルが下がる

→ 継続できる

💡 問題は「書くかどうか」ではない。「何を書くか」だ

書く価値はわかった。でも、書けない...

よくある失敗パターンと、その対策を先に見ておこう

→ 「あるある」を知れば、回避できる

よくある失敗パターン①

書けない・出せない

✕ 完璧主義で出せない

「もっと調べてから」「もっと良い文章に」「誰かに突っ込まれるかも」→ いつまでも下書きのまま

✓ 対策

80%で出す / 後から直せる（Webの良さ） / 批判より「参考になった」の方が多い

✕ ネタがない

「書くほどのことじゃない」「誰かがもう書いてる」「大したことしてない」

✓ 対策

あなたの言葉で書く価値がある / 同じ内容でも視点が違う / 「ハマったこと」は全部ネタ

💡 出さなければゼロ。出せば1。直せば1.1



よくある失敗パターン②

読まれない・伝わらない

✕ 情報の羅列で終わる

コードを貼っただけ / 公式ドキュメントの要約 / 「私」が不在 → 読者「で、何が言いたいのか？」

✓ 対策

「私はこう使った」「私はこう思った」を入れる / なぜその方法を選んだか / 失敗談も書く

✕ 結論がわからない

最後まで読まないとわからない / 「で、どうすればいいの？」

✓ 対策

最初に結論を書く / タイトルに結論を含める / 「TL;DR」セクションを設ける

💡 「私」を入れるだけで、記事は生きる

「AIに書かせればいいのか？」

その疑問に、先に答えておく

→ AIは強力なツールだが、使い方を間違えると逆効果



「AIに書かせればいいのかでは？」

その疑問に答えておく

よくある期待

- ChatGPTに書かせれば楽では？
- AIで記事を量産できるのでは？
- 時間がないからAIに任せたい

結論から言うと

AIは強力なツールだが、**使い方を間違えると逆効果**

→ 人間とAIの役割分担を明確にすれば、執筆効率は上がる

AIに記事を書かせるとは何か

「それは本当にあなたの記事なのか」

私の答えは明確だ。記事はほとんどAIに書かせている。しかし、価値の源泉は私にある。

すでにAIの支援を受けている

- 予測変換
- LSP (Language Server Protocol)
- コード補完
- スペルチェック

→ 生成AIは**その延長線上**にある

では、私は何を担っているのか

「身体性」を供給している

私が素材（身体性）を提供し、AIが構造化し、私がレビューして調整する

→ この協働が現代の「執筆」

💡 AIは編集者。価値の源泉は、あなたの「身体性」にある



身体性とは何か

知識が「情報」から「経験」へ変容する過程

Rustの所有権システムを学んでいる。The Bookを読み、概念は「理解した」つもり。しかしコードを書くと `cannot borrow as mutable ...`。「知っている」と「書ける」の間にある断絶——これが身体性の欠落。

断絶を越えた瞬間の記録

- なぜエラーになったか格闘した
- イテレータの内部構造に気づいた
- 腹落ちした瞬間があった

→ 「生きた知見」として伝達可能

AIには生成できないもの

- 私の代わりに試行錯誤すること
- 私の代わりにコンパイラに叱られること
- 私の代わりに悔しがること

→ 苦闘から理解への遷移

💡 AIは「生の体験」を読める文章に整える。編集者の仕事だ



「流暢な嘘」という罠

AIに丸投げすると何が起きるか

「AIを使った記事には価値がない」——この批判は、ある意味で正しい。問題は「AIを使ったこと」ではなく、「**検証**というプロセスが抜け落ちていること」にある。

AIの出力の厄介さ

- 文法的に完璧
- 論理の構成も美しい
- 嘘でもスルスル頭に入る
- 「もっともらしさ」に特化

検証なしで公開すると

- 不正確な情報の垂れ流し
- 読者の信頼を失う
- 自分の評判を傷つける
- ブレーキの効かない車

💡 LLMは確率で「次の単語」を選んでいるだけ。真偽への誠実さはない



AIを監視するのは人間の仕事

確率の波を制御し、事実という地面に杭を打つ

人間が担う「監修者」の役割

- 事実確認（コードは動くか？）
- 論理確認（説明に飛躍はないか？）
- 経験確認（自分の体験と一致するか？）
- 読者確認（誤解を招かないか？）

監修を効率化する方法

- 別のAIでクロスチェック
- 実際にコードを動かす
- 一晩寝かせてから読む
- 第三者に見せる

→ 「AIを監視するAI」も活用

💡 AIのエンジン出力に酔うな。冷静な「監修者」であり続ける



では、具体的にどう使うか

監修者としてのAI活用ガイドライン

✓ 頼んでいいこと（編集者として）

- 記事構成の相談
- 下書きのレビュー
- 文章のリライト
- タイトル案の生成
- 誤字脱字・事実チェック

→ 「相談」「壁打ち」の姿勢で

✗ 頼んではいけないこと（身体性の捏造）

- 経験していないことの執筆
- 技術的判断の丸投げ
- 動作確認なしのコード掲載
- 検証なしでの公開

→ 「書いて」は危険

💡 AIは編集者。「身体性」を供給するのはあなたの仕事



AIの出力には「私」がない

正確でも、読まれない文章になる

AIの出力の特徴

- 正しいけど、誰が書いても同じ
- 「～と言えるでしょう」「～が重要です」
- 具体的なエピソードがない
- 読んでも心に残らない

→ 身体性がない

だから手を入れる

- 「私は」を入れる
- 自分の失敗談を追加する
- 感情・気づきを入れる
- 自分の言葉で言い換える

→ 身体性を注入する



身体性の本質：「情報」と「経験」の違い

同じ事実でも、伝わり方がまったく違う

情報としての記述

「リソース制限は重要な設定項目です」

- 検索すれば出てくる
- 誰が書いても同じ
- 読んでも行動が変わらない

→ 右から左に流れていく

経験としての記述

「リソース制限をナメていた。OOMKilledで3時間溶かすまでは」

- あなたにしか書けない
- 痛みが伝わる
- 読者の行動が変わる

→ 記憶に残る

💡 情報を右から左に流すな。経験を通過させる



身体性の3つの要素

経験を言語化するための視点

① 苦闘 (Struggle)

何につまずいたか、何時間溶かしたか、どこで間違えたか

→ 読者も同じ穴に落ちる前に救える

② 転機 (Turning Point)

何がきっかけで理解できたか、どの瞬間に腹落ちしたか

→ 読者に追体験させられる

③ 教訓 (Lesson)

次から何を変えるか、どう予防するか

→ 読者の行動を変えられる

AIが整え、あなたが魂を吹き込む。これが現代の執筆だ



AIと引き算：捨てる勇気

AIは「足す」のが得意、人間は「引く」のが仕事

AIの特徴：AIに「〇〇について書いて」と頼むと、関連情報をすべて詰め込もうとする。結果、長くて焦点のぼやけた文章になりがち。

✕ AIに任せきり

- 網羅的だが冗長
- 優先順位が不明
- 「で、結局何？」になる

✓ 人間が引き算する

- 「この記事で伝えるのは3つだけ」
- 「この段落は削る」
- 「この例は別記事にする」

引き算のコツ：「これを削ったら記事が成り立たなくなるか？」答えがNoなら、削る。

💡 AIは素材を出す。何を残すか決めるのが、あなたの仕事



では、何を書くか？

「書くことがない」は思い込み。日常にネタは転がっている

→ ネタのを見つけ方と、発想を広げる6ステップを紹介します



日常からネタを発掘する技術

ブログネタは探すものではない、気づくもの

🤔 学習過程での「なぜ？」

理解するのに時間がかかった概念
直感に反する仕様は
記事になりやすいトピック

🔍 検索で見つからなかった問題

検索して出てこなかった答えを
あなたが書けば、次の誰かは3秒で見つかる

🔄 繰り返し説明していること

チーム内で何度も同じ説明をしている内容は
記事化する価値が高い

❓ 「あれ？」と思った瞬間

予想通りに動かなかった。なぜ？
意外な挙動を示した。なぜ？
この「なぜ？」が記事の種

でも、ネタはあっても形にならない

「書きたいこと」を「書ける記事」に変えるには？

よくある悩み

- ネタはあるけど、どう膨らませればいいのかわからない
- 書き始めても途中で手が止まる
- 「で、何が言いたいの？」と自分でも思う
- 構成がぐちゃぐちゃになる

必要なのは「発想の型」

ネタを記事に変えるには、体系的なアプローチが必要

→ 「選択の科学」で有名なシーナ・アイエンガー氏の
THINK BIGGERを紹介

6ステップで「書けない」を解消する

シーナ・アイエンガー氏とは

「ジャムの法則」で有名なコロンビア大学教授

ジャムの法則（2000年）

スーパーでジャムの試食ブースを設置

- **24種類**を並べた場合
- **6種類**を並べた場合

結果：24種類の方が足を止める人は多いが、
実際に購入したのは6種類の方が圧倒的に多かった

この実験が示すこと

選択肢が多すぎると、人は「選ぶ」という行為自体ができなくなる

→ **選択の科学**（2010年）で世界的ベストセラーに

→ その後10年かけて「選択肢すらない場合、どうすればいいか」を研究

💡 **THINK BIGGER（2023年）** は、その研究成果をまとめた13年ぶりの新作



THINK BIGGER：6ステップの発想法

「新しいものごとは、要素が新しいのではない。組み合わせ方が新しいのだ」

 この6ステップを「ブログの執筆」に応用します

天才のイノベーターが無意識に行っていた思考を、誰でも再現できる6ステップに分解

ガンディー、Netflix、ピカソ、コロナワクチン開発...歴史に残る「ビッグアイデア」は、ゼロからの発明ではなく「既存の要素の新しい組み合わせ」から生まれている。ブレストで量を出すのではなく、**質を追求**する発想法。

Step 1

課題を選ぶ

Step 2

課題を分解

Step 3

望みを比較

Step 4

箱の中と外

Step 5

選択マップ

Step 6

第三の目



「書けない」は才能の問題じゃない。この6ステップのどこかが詰まっているだけ



まずは「何を書くか」から

Step 1：課題を選ぶ

6ステップの最初は「テーマ選び」

「書きたいことはあるけど、何から手をつければ...」



まず「1記事で書けるサイズ」に絞る



Step 1：課題を選ぶ

一言で言うと：「何について書くか」を決める

このステップでやること → 記事のテーマを決める

ポイント：サイズ感が重要

- 大きすぎると書ききれない
- 小さすぎると内容が薄い
- 「1記事で完結する」サイズを探す

具体例で比較

- ❌ 「Kubernetesの使い方」 → 大きすぎ
- ❌ 「kubectl getコマンド」 → 小さすぎ
- ✅ 「本番障害を防いだリソース制限の設定」

💡 「誰かの役に立つ」かつ「自分が書ける」サイズを見つける



Step 1：よくある失敗パターン

テーマ選びでつまづく人の共通点

✗ 大きすぎるテーマを選ぶ

「Kubernetesについて」「AWSの使い方」

→ 書ききれない → 途中で挫折

✗ 小さすぎるテーマを選ぶ

「このコマンドのオプション」

→ 内容が薄い → 書く意味を見失う

✓ ちょうどいいテーマの見つけ方

「自分が2～3日かけて解決したこと」がベスト

→ 深みがある & 1記事で完結する



Step 1：課題のレベルを上げ下げする

ちょうどいいサイズを探す技術

レベルを上げる（抽象化）

「CI/CDパイプラインの高速化」

↑

「GitHub Actionsのキャッシュ設定」

より大きな視点で課題を捉え直す

レベルを下げる（具体化）

「GitHub Actionsのキャッシュ設定」

↓

「node_modulesのキャッシュで3分短縮した話」

より具体的に分解する

技術ブログの「おいしいサイズ」

- 1記事で完結する / 読者が再現できる / 自分の経験が活きる
- 上げ下げしながら「書きやすく、読まれやすい」ところを探す



テーマが決まった。次は「構成」

Step 1 → Step 2

Step 1 完了：書くテーマが決まった

「テーマは決まったけど、何から書けば...」



次は「何を、どの順番で書くか」を決める



Step 2：課題を分解する

一言で言うと：「記事の目次」を作る

このステップでやること → テーマを「書くべきこと」に分解する（5つまで）

なぜ分解するのか

「〇〇の話を書こう」だけでは手が動かない
→ 「何を、どの順番で書くか」を決める

分解 = 記事の骨格を作る

- 分解した項目がそのまま見出しになる
- 5つまでに絞ると焦点が定まる



Step 2：分解の具体例

テーマを見出しに分解する

テーマ：「OOMで落ちたPodを調査した話」

分解結果 (= 見出し)

1. 何が起きたか（症状）
2. どう調査したか（手順）
3. 原因は何だったか（分析）
4. どう解決したか（対策）
5. 再発防止（教訓）

💡 分解すると記事の構成が見えてくる



Step 2：分解の実践例

「OOM調査」をさらに分解してみる

最初の分解（大きい）

1. 症状
2. 調査
3. 原因
4. 解決
5. 再発防止

さらに分解（細かい）

1. 症状 → エラーログ、影響範囲
2. 調査 → コマンド、ツール
3. 原因 → 仮説、検証
4. 解決 → 設定変更、確認
5. 再発防止 → 監視設定

分解のコツ

- 最初は大きく分解 → 書きながら細かくする
- 「これは1段落で書ける？」が目安
- 細かくしすぎると読みにくい → バランスが大事



Step 2：なぜ「5つまで」なのか

ジャムの法則をここでも活かす

選択肢が多すぎると選べない

ジャムの法則と同じ。サブ課題を10個も20個も出すと、どれに注力すべきか分からなくなる。

5つに絞る効果

- 本当に重要なものを見極められる
- 各サブ課題に十分な時間を割ける
- 記事の構成がシンプルになる

絞る過程で見えてくるもの

「これは本当に必要か？」と自問すると...

- 実は同じことを言っている項目
- なくても成立する項目
- 別の記事にした方がいい項目

→ 削ることで本質が見える

⚠ 「網羅的に」より「絞って深く」の方が読者に刺さる



構成ができた。次は「視点」

Step 2 → Step 3

Step 2 完了：記事の構成（目次）ができた

「構成は決まった。でも、どんなトーンで書けば...」



次は「誰の視点で書くか」を決める



Step 3：望みを比較する

一言で言うと：「誰のために書くか」を決める

このステップでやること → 「自分・読者・会社」の望みを整理し、優先順位をつける

3つの望み

1. **自分** → 書きたいこと、伝えたいこと
2. **読者** → 知りたいこと、解決したいこと
3. **会社** → 宣伝、採用、ブランディング

結論：自分を優先してOK

全部満たそうとすると薄くなる。

まず自分が書きたいことを書く

→ 熱量のある記事は読者にも伝わる

💡 「誰かのために」より「自分のために」書いた記事の方が刺さる



Step 3：具体例で考える

同じテーマでも「誰のため」で変わる

テーマ：「GitHub Actionsでテストを高速化した話」

自分の望み優先の記事

- 試行錯誤の過程を詳しく
- 失敗した方法も書く
- 自分の学びを残す

→ 技術的に深い記事に

読者の望み優先の記事

- 結論を先に
- コピペで使える設定
- 手順だけシンプルに

→ 実用的な記事に

→ どちらも正解。**今回どちらで書くか**を決めるのがこのステップ



Step 3：望みの対立を認める

全部は満たせない、だから選ぶ

よくある対立

- 自分：詳しく書きたい
- 読者：早く結論を知りたい
- 会社：会社の宣伝を入れてほしい

全部を満たそうとすると...

→ 長くて読まれない記事になる

選択する勇気

- 今回は「自分の学び」を優先
- 今回は「読者の課題解決」を優先
- 今回は「会社への貢献」を優先

1記事1テーマで強弱をつける

⚠ 全方位に良い顔をしようとする、誰にも刺さらない



視点が決まった。次は「材料集め」

Step 3 → Step 4

Step 3 完了：誰のための記事が決まった

「方向性は決まった。でも、内容をどう膨らませれば...」



次は「参考になる事例」を集める



Step 4：箱の中と外を探す

一言で言うと：「書く前に下調べ」をする

このステップでやること → 記事の材料になる情報を集める

箱の中（同じテーマ）

- 公式ドキュメント
- 他の人の同テーマの記事
- GitHub Issues、Stack Overflow

→ 正確性を担保する・抜け漏れ防止

箱の外（自分の経験）

- 実際に試した結果
- ハマったポイントと解決策
- 自分なりの工夫や改善

→ オリジナリティの源泉



「調べた情報」 + 「自分の経験」 = 価値のある記事



Step 4：下調べの具体的な方法

どこで何を調べるか

正確性のための調査

- 公式ドキュメントで仕様を確認
- 他の記事と自分の理解を照合
- バージョン情報を確認
- 最新の情報かどうかチェック

→ 間違いを書かないために

差別化のための整理

- 他の記事にない視点は何か？
- 自分だけが経験したことは？
- より詳しく書ける部分は？
- 図解できる部分は？

→ 読む価値を作るために



「正しさ」と「独自性」の両方を意識して材料を集める



Step 4：よくある下調べの失敗

これをやると記事の質が下がる

✕ 下調べ不足

- 公式ドキュメントを読まずに書く
- 古い情報をそのまま引用
- 自分の環境でしか試していない

→ 間違った情報を広めてしまう

✕ 下調べしすぎ

- 完璧に理解してから書こうとする
- 他の記事を読みすぎて書けなくなる
- 「もう書かれてる」と諦める

→ いつまでも書き始められない



「正確性を担保できる程度」の下調べでOK。完璧を目指さない

Step 4：他の記事との差別化

「既に書かれてる」は気にしない

同じテーマでも価値が出せる理由

- 環境が違う（OS、バージョン、構成）
- 文脈が違う（なぜそれをしたか）
- 深さが違う（どこまで掘り下げるか）
- 切り口が違う（何を強調するか）

あなたの記事にしかない価値

- **あなたの環境**で動いた事実
- **あなたがハマったポイント**
- **あなたの言葉**での説明
- **あなたの図解**や整理

→ これが読者にとっての価値



「n番煎じ」でも、あなたの経験を加えれば価値になる



材料が揃った。次は「選択」

Step 4 → Step 5

Step 4 完了：参考事例と表現方法を集めた

「材料は揃った。でも、どれを使えばいい...？」



次は「選択マップ」で情報を整理し、何を書くか決める



Step 5：選択マップ

一言で言うと：「集めた情報を整理して、何を書くか選ぶ」

このステップでやること → 調べた情報を「選択マップ」で整理し、記事に使うものを決める

選択マップとは

THINK BIGGERで紹介されている思考ツール

集めた選択肢を視覚的に整理し、最適な組み合わせを見つける方法

なぜ必要か

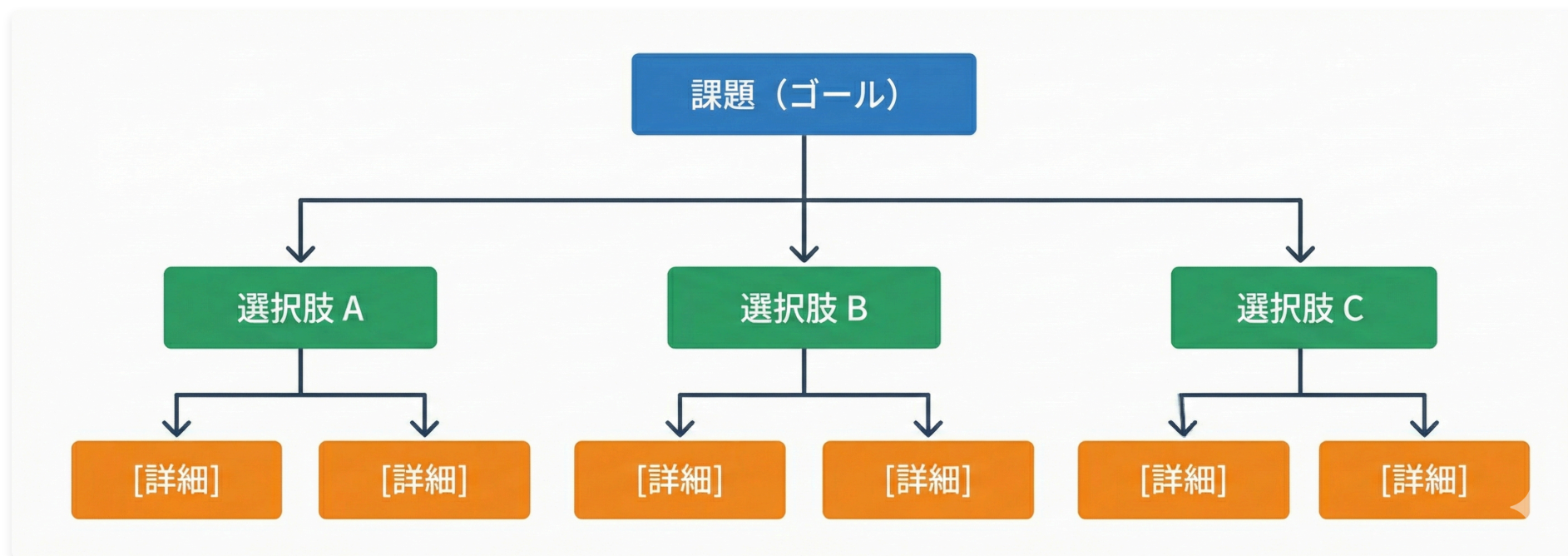
- 調べた情報が多すぎて迷う
- 何を書くべきか決められない
- 全部詰め込むと散漫になる
- 「選ぶ」ことで記事が締まる



Step 5：選択マップの考え方

THINK BIGGERでの選択マップ

選択マップの基本構造



ポイント：課題から分岐して選択肢を並べ、各選択肢のメリット・デメリットを可視化する

Step 5：選択マップの目的

「何を選ぶか」を明確にする

選択マップが解決する問題

- 調べた情報が多すぎる
- どれも良さそうに見える
- 判断基準が曖昧
- 「全部書く」は読者に不親切

選択マップで得られること

- 選択肢の全体像が見える
- 比較軸が明確になる
- 「なぜこれを選んだか」が説明できる
- 記事の焦点が定まる

💡 選択マップは「選ぶ理由」を可視化するツール



Step 5：技術ブログでの選択マップ

ブログ記事で「選ぶ」もの

技術ブログで選択マップを使う場面

記事の種類	選択マップで整理するもの
ライブラリ比較	どのライブラリを推奨するか
トラブルシュート	どの解決策を紹介するか
設計パターン	どのアプローチを詳しく書くか
導入記事	どの手順・設定を載せるか

Step 5：選択マップを使う例

「OOMKilled 調査」記事の場合

調査方法は複数ある

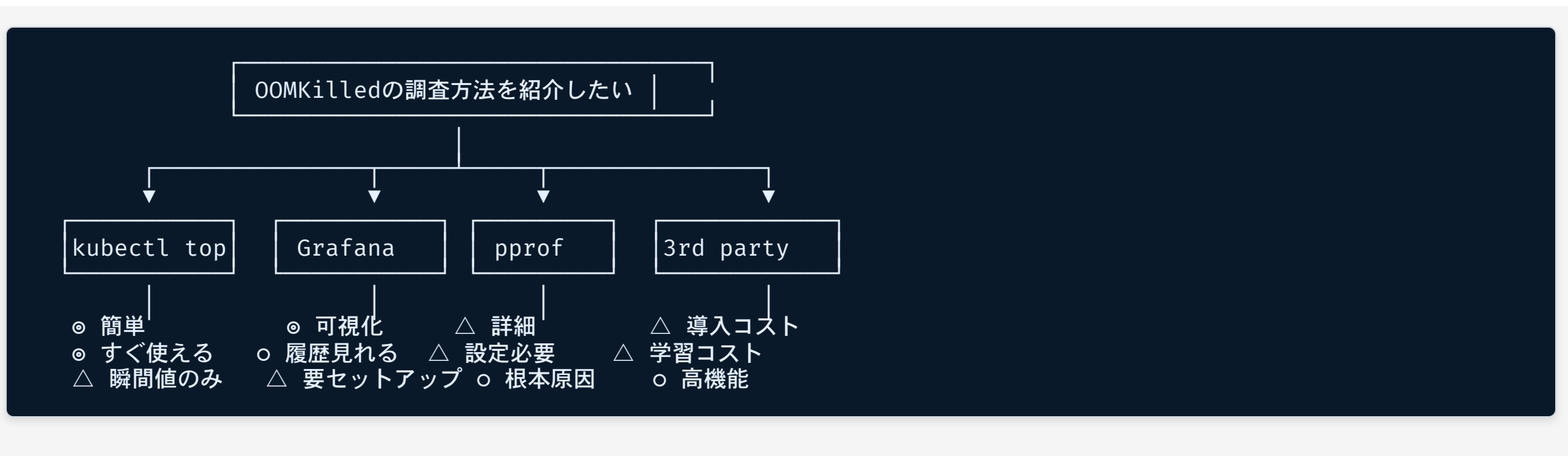
- kubectl top
- Grafana
- pprof
- 3rd party tool

読者に最も役立つのはどれか？ を選択マップで整理する



Step 5：選択マップの具体例

「OOM調査方法」の選択マップ



選択結果：読者の多くは「まず何が起きてるか知りたい」

→ **kubectl top + Grafana** を中心に書く（pprof は発展編として軽く触れる）

Step 5：選択マップから記事構成へ

選んだものを「どう並べるか」

選択マップの結果

- kubectl top：◎（簡単、すぐ使える）
- Grafana：○（履歴が見える）
- pprof：△（上級者向け）

→ **kubectl top + Grafana** を中心に書く

記事構成に落とし込む

1. 問題発生（何が起きたか）
2. kubectl top で現状確認
3. Grafana で推移を可視化
4. 原因特定と解決
5. （発展）pprof で深掘り

💡 選択マップで「選んだもの」を軸に記事構成を作る



Step 5：選択マップのメリット

「なぜこれを書いたか」が説明できる

選択マップなしで書くと

- 「なんとなく」で内容を決める
- 後から「あれも書けばよかった」
- 読者から「なぜこれだけ？」と思われる
- 自分でも選んだ理由が説明できない

選択マップで整理すると

- 選択肢を網羅した上で選べる
- 「読者にとって最適だから選んだ」と言える
- 記事の冒頭で説明できる
- 書いてないことへの言い訳もできる

💡 選択マップは「選ばなかったものへの説明責任」も果たせる



Step 5：技術ブログでの現実的なやり方

選択マップは「頭の中で」やることも多い

実際の執筆では

- 選択マップを図に描かないことも多い
- **まず全部書いてみる**
- 書いた後に「これは要らない」と削る
- 結果的に選択が行われる

大事なのは「選ぶ意識」

- 全部詰め込まない
- 読者にとって最適なものを選ぶ
- 書いた後に見直して削る勇気
- **「書かない」も選択**

💡 選択マップは考え方。図に描くかどうかは自由

記事構成が決まった。最後は「検証」

Step 5 → Step 6

Step 5 完了：選択マップで情報を整理し、記事構成を決めた

「書き終わった。でも、これで大丈夫？」



最後は「自分以外の目」でチェックする



Step 6：第三の目で検証する

一言で言うと：「一晩寝かせてから見直す」

このステップでやること → 時間を置いて、または他者の視点で記事をチェックする

方法①：次の日の朝に読む

- 書いた直後は「完璧」に見える
- 一晩寝ると冷静に読める
- 「なんでこんな書き方した？」が見つかる

→ 時間が「第三の目」になる

方法②：AIにチェックしてもらう

- ChatGPT / Claude に記事を貼る
- 「わかりにくい部分を指摘して」
- 「誤字脱字をチェックして」

→ 24時間使える校閲者

💡 「書いた直後に公開」は危険。最低でも翌朝チェック



Step 6：なぜ検証が必要か

書いた本人には見えない穴がある

書いた本人あるある

- 「当然わかるでしょ」と省略
- 専門用語を説明なしで使う
- 論理の飛躍に気づかない
- 誤字脱字を見落とす

→ 自分では完璧に見える

読者の視点で見ると

- 「なんでこうなるの？」
- 「この用語わからない」
- 「話が飛んでる」
- 「誤字があると信頼度↓」

→ 書いた本人には見えない



「自分以外の目」を通すだけで、記事の質が劇的に上がる

Step 6：最終確認

技術ブログならではのチェックポイント

技術的な正確性

- コードは実際に動くか？
- バージョン情報は正しいか？
- コマンドの出力は正確か？
- リンク切れはないか？

→ 動かないコードは信頼を失う

読みやすさ

- 専門用語に説明はあるか？
- コードブロックは見やすいか？
- 図や画像は適切か？
- 長すぎるセクションはないか？

→ 公開前に一度通して読む

💡 未来の自分が読んだ時に「わかりやすい」と思えるか



6ステップのまとめ①

どこで詰まっているかを特定する

Step 1が不足

課題が大きすぎ/小さすぎ

- レベルを上げ下げ
- 「書ける」サイズに

Step 2が不足

課題が漠然としている

- サブ課題に分解
- 5つまでに絞る

Step 3が不足

誰向けか曖昧

- 望みを書き出す
- 優先順位をつける

Step 4が不足

視点が狭い

- 領域外の事例を探す
- 成功事例に注目

Step 5が不足

構成が決まらない

- まず全部書いて削る
- 「選ぶ意識」を持つ

Step 6が不足

独りよがりになる

- 他者に説明する
- 説明し返してもらう



6ステップのまとめ②

選択の連続で記事は生まれる

THINK BIGGERの本質：「選ぶ」ことの連続

課題を選ぶ → サブ課題を選ぶ → 望みを選ぶ → 戦術を選ぶ → 組み合わせを選ぶ → 検証で磨く

 選ぶ →

 分解 →

 比較 →

 探す →

 マップ →

 検証

💡 「書けない」は才能の問題じゃない。この6ステップのどこかが詰まっているだけ

6ステップの本質：引き算の技術

「足す」より「削る」が難しい

THINK BIGGERの6ステップは、すべて「引き算」のプロセス

Step 1-2

無数のテーマから1つを選ぶ
→ 他を**捨てる**決断

Step 3-4

読者の望みを絞る
→ 全員に届けようとししない

Step 5-6

選択肢を比較して削る
→ 最良の1つに絞る

✗ 足し算の発想

あれもこれも詰め込む → 焦点がぼやける → 誰にも刺さらない

✓ 引き算の発想

「これだけは伝える」を決める → 残りは捨てる → 刺さる記事になる

💡 良い記事は「何を書かないか」で決まる



ネタは見つかった。では、どう書くか？

記事の種類を選び、型に沿って書けば迷わない

→ 記事タイプ・タイトル・導入文・本文の型を紹介します



技術ブログの種類を考える前に

「何を伝えたいか」で種類が決まる

技術ブログは大きく **2つの軸** で分類できる

軸①：時間軸

- **過去**：何が起きたか（障害、バグ、失敗）
- **現在**：今どうしているか（運用、設計）
- **未来**：これからどうなるか（予測、展望）

軸②：抽象度

- **具体**：手順、コード、設定（How）
- **中間**：設計、判断、比較（What/Why）
- **抽象**：原則、哲学、教訓（Insight）

💡 この2軸の組み合わせで、記事の種類が決まる



技術ブログの種類①

学習ログ（過去 × 具体）

新人に特におすすめ 🌱 新しい技術を学んだ過程を記録

なぜ価値があるか

- 公式ドキュメントにない「つまずき」がある
- 同じ道を歩む人の時間を節約できる
- 書いた本人の理解も深まる

構成例

1. 学習の動機
2. 前提知識と環境
3. つまずきと解決
4. 得られた気づき

💡 英語チュートリアルを日本語でやってみた、だけで価値がある



技術ブログの種類②

トラブルシューティング記事（過去 × 中間）

障害・バグを追跡し、解決するまでの思考過程を共有

なぜ価値があるか

- 同じ症状で困っている人を直接救える
- 「思考の過程」は検索で出てこない
- 間違った仮説も含めて共有することで学びが深まる

構成例

1. 何が起きたか（症状）
2. 最初の仮説（間違いも含む）
3. 原因の特定過程
4. 解決策と再発防止

💡 「この方法じゃダメだった」は「この方法でいけた」と同じくらい価値がある

技術ブログの種類③

設計・意思決定記事（現在 × 中間）

なぜその設計にしたのか、どう判断したのかを共有

なぜ価値があるか

- 「正解」だけでなく「なぜ他を選ばなかったか」が学びになる
- 同じ判断を迫られた人の参考になる
- 自分の判断を言語化することで整理される

構成例

1. 解決したい課題
2. 検討した選択肢
3. 比較と評価軸
4. 最終的な判断と理由
5. 今後の懸念点



「なぜAではなくBを選んだか」が最も読者の役に立つ

技術ブログの種類④

教訓・振り返り記事（過去 × 抽象）

経験から抽出した一般化可能な教訓を共有

なぜ価値があるか

- 具体的な経験を抽象化することで汎用的な学びになる
- 失敗談は成功談より学びが深い
- 「思っていたのと違った」は最も価値のある情報

構成例

1. 状況と背景
2. 当初の想定
3. 実際に起きたこと
4. なぜ想定と違ったか
5. 得られた教訓

💡 「～だと思っていたけど実際は違った」 ← 読者が最も求めている情報



技術ブログの種類⑤

手順書・ガイド記事（現在 × 具体）

環境構築やセットアップ手順を詳細に解説

なぜ価値があるか

- 「動く状態」までの最短経路を示せる
- 公式ドキュメントの行間を埋められる
- 半年後の自分を救える

構成例

1. 前提条件と環境
2. インストール手順
3. 設定と確認
4. よくあるエラーと対処

💡 コピペで動く状態まで持っていく。それが最大の価値



技術ブログの種類⑥

比較・評価記事（現在 × 中間）

複数の選択肢を比較し、評価基準を示す

なぜ価値があるか

- 選択に迷っている人の判断材料になる
- 「どれも良さそう」を「この場合はこれ」に変換できる
- 評価軸を示すことで読者の思考を助ける

構成例

1. 比較の目的と前提
2. 比較対象の概要
3. 評価軸の設定
4. 各項目の評価
5. 結論と推奨

💡 「どれがいいか」ではなく「どういう場合にどれがいいか」を示す



記事の種類が決まったら、次は「見せ方」

タイトルと構成で読まれるかどうかが決まる

同じ内容でも、見せ方で反応が変わる

- タイトルで読むか読まないかが決まる
- 導入文で読み続けるかが決まる
- 本文の構成で理解度が決まる

これから紹介すること

- タイトルの付け方（テンプレート）
- 導入文の書き方（型）
- 本文の構成パターン
- コードブロック・図解の活用

タイトルの付け方①

タイトルの本質：「約束」と「期待値」のコントロール

タイトルとは何か

タイトルは「読者との約束」である。クリックした読者に対して「この記事を読めば、これが得られる」という**期待値**を設定する役割を持つ。

期待値が高すぎると

- 内容が伴わないと「釣り」と思われる
- 信頼を失い、次から読まれなくなる
- SNSで批判されるリスク

期待値が低すぎると

- そもそもクリックされない
- 熱量や価値が伝わらない
- 埋もれてしまう

💡 良いタイトルは「約束を少しだけ超える」記事とセットになる



タイトルの付け方②

タイトルの3要素：「対象」×「行動」×「結果」

対象（What）

技術名、ツール名、問題

行動（How）

導入した、解決した、学んだ

結果（So What）

速度向上、エラー解消

例：「**Next.js 14**に移行して表示速度が**3倍**になった話」

タイトルの付け方③

タイトルには「強度」がある

強度とは、タイトルが主張する**確信度の高さ**のこと。

強度	表現パターン	特徴
弱	「〇〇のメモ」「〇〇を試した」	謙虚だが埋もれる
中	「〇〇した話」「〇〇で学んだこと」	バランスが良い
強	「〇〇すべき理由」「〇〇は間違い」	主張が強い

タイトルの付け方④

強度の選び方

強いタイトルが許される時

- 数値的な成果がある
- 実際に問題を解決した
- 長期間の実運用経験

弱めにすべき時

- 初めて試した技術
- まだ検証途中
- 環境依存の可能性

💡 確信があるほど、強いタイトルを使う権利がある

タイトルの付け方⑤

テンプレートを活用する

体験・実践系

- 「〇〇で△△した話」
- 「〇〇を導入して□□になった」

問題解決系

- 「〇〇エラーの原因と解決方法」
- 「〇〇できない時の対処法」

入門・解説系

- 「〇〇入門：△△から始める」
- 「初心者向け〇〇の始め方」



タイトルの付け方⑥

まとめ：タイトルは「約束」である

3つの原則

1. 3要素を含める（対象・行動・結果）
2. 強度を意識する
3. 約束を守る

実践のコツ

- 本文を書き終えてから確定
- 迷ったら「中」の強度から
- 数字や技術名を入れる

これらはあくまで**守破離の「守」**。型を身につけたら、自分のスタイルを見つけていく。



タイトルで興味を引いた。次は導入文

導入文の本質：「期待の確認」と「信頼の獲得」

導入文とは何か

タイトルで設定した「約束」を確認し、「この記事は読む価値がある」と信じてもらうためのもの。

タイトル：約束を提示する

導入文：約束を確認させる

導入文の書き方①

導入文で読者の心を掴む

スマホでぱっと見たとき、**最初の3～5行**が一瞬で目に入る。そこで掴めなければ離脱する。ワンスクロールまでに文体や雰囲気から「読む価値があるか」を察する。

関連性

「これは自分の問題か？」

信頼性

「この人は信頼できるか？」

見通し

「何が得られるか？」

💡 3～5行 + ワンスクロールで勝負が決まる

導入文の書き方②

導入文の3要素

問題提示

読者の課題を言語化する

「〇〇で困った」

「〇〇がわからない」

解決予告

何を伝えるかを示す

「原因と解決策を共有」

「手順を解説」

信頼の根拠

なぜ自分が書けるか

「2日調査した」

「本番運用で検証」



導入文の書き方③

信頼性の示し方：謝罪ではなく限定

✗ 謝罪で信頼を損なう

- 「初心者なので間違いあるかも」
- 「間違ってたらすみません」
- 「備忘録です」

→ 読む価値があるか不安になる

✓ 限定で誠実さを示す

- 「私の環境（Ubuntu 22.04）で検証」
- 「執筆時点（2025年12月）の情報」
- 「〇〇については扱いません」

→ 信頼性が上がる

💡 謙虚さは「限定」で表現する。謝罪で表現しない

導入文の書き方④

テンプレートを活用する

問題解決型

「〇〇でハマった。原因は△△。調査過程と解決策を共有する」

学習記録型

「〇〇を学んだ。つまづいたポイントと理解の過程を共有する」

これらも**守破離の「守」**。型を身につけたら、自分の声で書く。

本文の構成パターン①

問題解決型（最も汎用的）

構成：問題 → 調査 → 解決 → 結果

バグ修正、トラブルシューティング、エラー解消に最適

1. **問題**：症状・発生状況・影響範囲

2. **調査**：仮説→試行錯誤→原因特定

3. **解決**：解決策・コード・理由

4. **結果**：解決後の状態・再発防止・学び

💡 迷ったら「問題解決型」から始めよう。最も読者ニーズが高い



本文の構成パターン②

チュートリアル型・比較検討型

チュートリアル型

構成：完成像 → 手順 → 確認

環境構築、セットアップに最適

- まず何ができるようになるか
- ステップバイステップで説明
- 動作確認の方法

比較検討型

構成：背景 → 選択肢 → 比較 → 結論

技術選定、ツール比較に最適

- なぜ比較が必要になったか
- 評価軸と比較結果
- 選んだ理由と根拠

本文の構成パターン③

振り返り型・概念解説型

振り返り型

構成：実施 → 良かった点 → 改善点

経験談、プロジェクト振り返りに最適

- 何をやったかの概要
- うまくいったこと / 改善すべきこと
- 次回への教訓

概念解説型

構成：What → Why → How

技術解説、入門記事に最適

- それは何か（定義）
- なぜ必要か（背景）
- どう使うか（実例）



コードブロックの使い方①

コードの見せ方で記事の質が決まる

✖ 悪い例

コードだけ貼る

```
resources:
  limits:
    memory: "128Mi"
    cpu: "500m"
```

- 何のコードかわからない
- なぜこの値かわからない
- コピペしても意味がわからない

✔ 良い例

前に説明 → コード → 後に補足

「Podのメモリ上限を128MiBに設定します」

```
resources:
  limits:
    memory: "128Mi" # これを超えるとOOMKilled
    cpu: "500m" # 0.5コア分のCPU
```

「128MiBは小さめの値です。アプリの実際の使用量に合わせて調整してください」

💡 コードは「何を」「なぜ」「どう使うか」とセットで



コードブロックの使い方②

実践的なルール

長さのルール

- **理想**：20行以内 / **許容**：50行まで
- **それ以上**：分割する
- 長いコードは読まれない

コメント・その他

- 重要な行に「なぜ」を説明
- 実行結果も載せる（動作の証拠）
- 言語指定を忘れずに（ハイライト）



図解とスクリーンショットの活用①

図解を入れるべき場面

アーキテクチャ図

システム構成、コンポーネント間の関係
→ 文章だけでは伝わらない

フロー図

データの流れ、処理の順序
→ シーケンスが重要な場合

スクリーンショット

GUI操作手順、エラーメッセージ
→ 「この画面のこのボタン」が一目でわかる

比較表・グラフ

パフォーマンス比較、機能比較
→ 数値の違いが視覚的にわかる

 「文章で説明するより図の方が早い」と思ったら図解する

図解とスクリーンショットの活用②

おすすめツールと図解のコツ

おすすめツール

- **CleanShot X** (Mac) 注釈・GIF録画
- **draw.io** (無料、多機能)
- **Excalidraw** (手書き風)
- **Mermaid** (コードで書ける)
- **Napkin AI** (AIで自動図解)
- **生成AI** (Gemini等で画像生成)

図解のコツ

- 色は3色まで / 矢印の向きを統一
- 余白を十分にする
- 文字は読める大きさ (最低12px)
- キャプションを必ず付ける

読者視点で書く

「自分が分かっていること」は読者には分からない

読者との「情報の非対称性」を埋める

書き手が知っていることを、読者は知らない

情報の非対称性とは

あなたは問題の背景、試行錯誤の過程、最終的な解決策をすべて知っている。しかし読者は何も知らない。この「知識の差」を意識せずに書くと、読者は置いてけぼりになる。

書き手の頭の中

- 前提知識がある
- 試した順番を覚えている
- なぜその結論に至ったか分かる

読者の頭の中

- 前提知識がない（かもしれない）
- 何を試したか知らない
- なぜそれが正解か分からない

💡 「自分が当たり前知っていること」を丁寧に書く。それが読者への誠実さ



情報の非対称性を埋める方法

読者の立場に立って書く

✕ 非対称性を無視した書き方

- 「〇〇を設定すればOK」
- 「当然△△を使います」
- 「ここは省略します」
- いきなり解決策から書く

✓ 非対称性を埋める書き方

- 「〇〇を設定します。なぜなら～」
- 「△△を使う理由は～」
- 「前提として□□が必要です」
- 問題→背景→解決策の順で書く

実践チェックリスト

- ☐ 専門用語に説明を添えたか？
- ☐ 「なぜそうするのか」を書いたか？
- ☐ 前提条件を明示したか？
- ☐ 読者が再現できる情報を書いたか？



読者を惹きつける記事構成

ストーリーで読ませる

読者は「物語」を求めている

単なる情報の羅列ではなく、あなたの「物語」を共有する

1. 問題提起

何に困ったか

2. コンテキスト

状況説明

3. 探求の旅

試行錯誤

4. 発見と学び

解決・気づき

5. 次のステップ

応用・展望

💡 失敗したアプローチも正直に書くと信頼性が上がる

読みやすさを高める実践テクニック

流し読みしやすい記事を目指す

最初の3行で掴む

- 問題提起や具体的な価値を示す
- 「この記事を読むとわかること」を明示

段落を短く

- 1段落3～4行まで
- 空行を恐れない

見出しを上手に使う

- 見出しだけ読んで概要把握可能に
- h2で大きな流れ、h3で詳細

箇条書きを活用

- 3つ以上の項目は箇条書き
- 視覚的に流し読みしやすい

書けた。でも、公開が怖い

批判されたらどうしよう...という不安への対処

公開をためらう理由

- 「間違ってたら叩かれるかも」
- 「マサカリが飛んでくるかも」
- 「炎上したらどうしよう」

→ 実際、批判より「参考になった」の方が圧倒的に多い

でも、備えは必要

批判を完全に避けることはできないが、**書き方次第でリスクは下げられる**

→ 「防御力の高い」書き方を紹介

防御力の高いブログを書く①

炎上しない・批判されにくい書き方

✖ 防御力が低い表現

- 「XXはYYより優れている」
- 「これが正しい方法だ」
- 「〇〇はオワコン」

→ 反例を出されて炎上リスク

✔ 防御力が高い表現

- 「私のケースでは」
- 「〇〇の条件下では」
- 「2025年12月時点では」

→ 経験の共有は反論しにくい

💡 断言は反発を招く。経験の共有は対話を生む

防御力の高いブログを書く②

具体的なテクニック

🎯 限定する・根拠を示す

- 「私の環境では」「執筆時点の情報」
- 「公式ドキュメントによると」
- 出典のリンクを貼る

🤔 逆の視点・推奨の使い分け

- 「一方で〇〇という意見もある」
- 「おすすめ」→「私は〇〇を使った」
- 主語を「私」にする

💡 ディスるな、推せ。批評は検証ログとして社内や小さなコミュニティで共有

公開前チェックリスト①

内容のチェック

基本事項

- ☐ タイトルは具体的か
- ☐ 導入文で何がわかるか明示
- ☐ 結論・まとめがあるか

技術的内容

- ☐ コードは動作確認済みか
- ☐ 環境情報あるか
- ☐ 実行結果を載せたか

 動かないコードを載せると信頼を失う。必ず手元で確認

公開前チェックリスト②

読みやすさとリスク回避

👁️ パッと見で読みやすいか

- ☐ 見出しだけで流れがわかる
- ☐ 段落は短く（3～4行）
- ☐ 視覚的要素は適切か
- ☐ 一晩寝かせて読み返したか

⚠️ 公開して大丈夫か

- ☐ **情報漏洩**：機密・APIキー・顧客情報
- ☐ **防御力**：断定表現を避け限定しているか
- ☐ **誤字脱字**：textlintを実行する

💡 一晩寝かせると、見えなかった粗が見える

書き方はわかった。あとは実践あるのみ

どこで書くか、いつ書くか、そしてよくある落とし穴

→ 続けるためのコツを紹介します

どこに書くか①

プラットフォームの選び方

Zenn / Qiita

- 技術記事に特化したプラットフォーム
- 機能はほぼ同じ、設計思想・運営思想に違いあり
- マークダウンで書ける、検索流入が多い
- 初心者が始めやすい

→ どちらが合うか、調べて選ぼう

はてなブログ（私の選択）

- 尊敬するまつもとりーさんが使っていた
- デザインの自由度が高い
- はてなブックマークとの相性◎（拡散力）
- テックブログ専用機能は少なめ

→ 憧れの人と同じ場所で書くのも良い

💡 初心者はZennかQiitaで始めよう。両方に投稿してもOK



どこに書くか②

その他の選択肢

個人ブログ（Hugo/Astro等）

- デザイン・機能を完全にコントロール
- SEO・収益化は自己責任
- 技術的挑戦も楽しみたい人に

会社の技術ブログ

- 会社名で信頼性UP・採用広報にも
- レビュー体制あり・業務時間で書ける場合も
- 注意：機密情報・退職後の扱い

Tips: はてブのコメントが気になる人へ

`<meta name="Hatena::Bookmark" content="nocomment" />` をサイトに入れるとコメント非表示にできる

 会社ブログと個人ブログ、両方やるのがベスト

場所は決まった。では、いつ書くか

執筆の本質：「記憶の鮮度」と「習慣の力」

「いつ書くか」の2つの原則

執筆タイミングは「記憶の鮮度」と「習慣の力」で決まる。この2つを最大化する戦略を選ぶ。

記憶の鮮度

体験から時間が経つほど、細部が失われる。「あの時どうやったっけ」が増える。

習慣の力

意志の力には限界がある。仕組みで書く状況を作る方が持続する。

いつ書くか①

記憶の鮮度を活かすタイミング

問題解決直後

- 記憶が最も鮮明
- 感情が乗りやすい
- 「次の人のため」という動機

→ 最強のタイミング

業務終了直後

- 「今日やったこと」が新鮮
- 振り返りとして機能
- 明日の自分への引き継ぎ

週末のまとまった時間

- 平日のメモを記事化
- 推敲・編集の時間が取れる
- 鮮度は落ちるがメモで補う

💡 鮮度が高いほど、書くコストが下がる

いつ書くか②

習慣の力を活かす仕組み

トリガーを決める

既存の習慣にくっつける

- 「PRをマージしたら書く」
- 「コーヒーを入れたら書く」
- 「電車に乗ったらメモする」

ハードルを下げる

完璧を求めない

- 「1行だけ」から始める
- 下書きのまま保存OK
- 「見出しだけ」でも勝ち

仕組みで強制する

意志に頼らない

- 朝15分を固定枠に
- カレンダーに予約
- 仲間と約束する

💡 継続の敵は「完璧主義」と「孤独」

いつ書くか③

鮮度を保存する：コーディングエージェントをネタ帳にする

記憶の鮮度は時間とともに失われる。その場でメモを残す仕組みがあれば、後から記事化できる。

やり方

- 日報用のSlash Commandを作成
- 作業の区切りで `/daily` を実行
- 学んだこと・ハマったことを記録
- 後で見返してブログネタに

メリット

- 記憶が新鮮なうちに残せる
- コード作業の流れを止めない
- AIが要約・整理を手伝う
- 蓄積が資産になる



参考書籍

もっと深く学びたい人へ

Writing for Developers

技術ブログの書き方に特化した洋書。アイデア出しから執筆・改訂まで全工程をカバー。技術者とライターとの共著で実践的

三行で撃つ 〈善く、生きる〉ための文章塾

近藤康太郎著。「最初の三行で読者を撃て」「短く、近く、シンプルに」。常套句禁止、熱量を込める。厳しいが効く

言語化するための小説思考

小川哲著。直木賞作家が「伝わる言葉」の生み出し方を開陳。読者との情報の非対称性をどう埋めるか

💡 技術だけでなく「書くこと」自体を学ぶと、記事の質が変わる



まとめ：今日から、書け

人との差異は武器

80%で出せ

ディスるな、推せ

AIと共に書け

まず自分のために書け。結果として、それが誰かを救う

ありがとうございました

おい、テックブログを書け。今週、1本。

@nwiizo | <https://3-shake.com>